RSA Encryption — 1970s

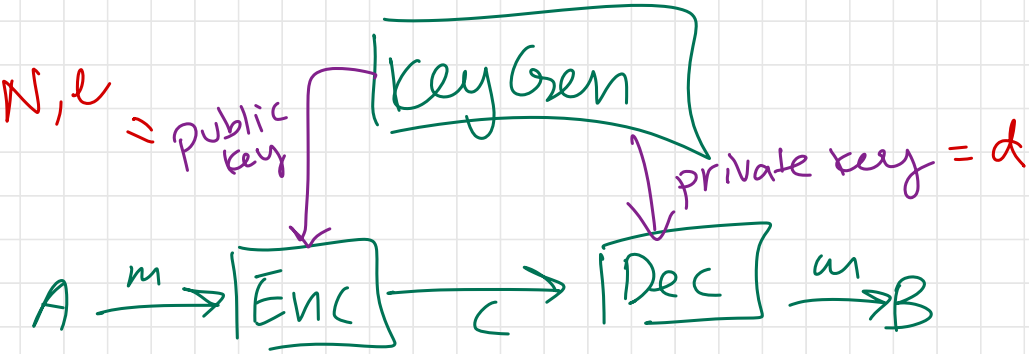
Send n-bit message m

An encryption scheme is 3 algs:

KeyGen

$Enc(k_1, m)$

$Dec(k_2, m)$



That satisfy 2 props:

Correctness: $Dec(k_2, Enc(k_1, m)) = m$ $\forall m, k_1, k_2$

security: everything efficiently computable about m with c is also efficiently computable without c.

Symmetric key: A, B share k

Public key: key has 2 parts: public
private

One-way functions

easy to compute; hard to invert

ex compute $p \cdot q = N$ for large primes p, q

for n-digit #s p, q

$O(n^{\log_2 3}) = O(n^{1.585})$

$O(n \log n)$ FFTs

vs. find p, q given N

naive getpq (N):

while i doesn't divide N: } 2 to $\sqrt{N}$

keep increasing i

$p = i$

$q = N/p$

runtime? $\sqrt{N}$ · fast ...

$N = 2^n$ ← length of N

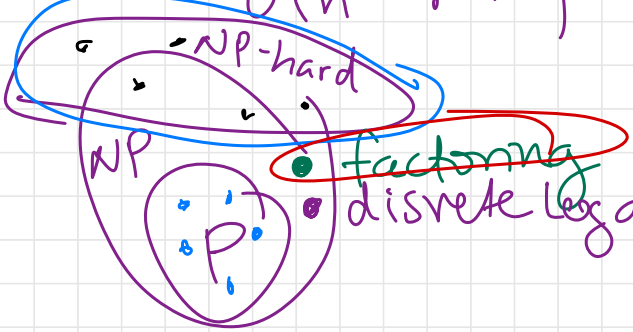
$O(\sqrt{N})$

$O(\sqrt{2^n}) = O((2^n)^{1/2}) = O(2^{n/2})$

best-known:

Generalized Number Field Sieve

$O\left(n^{\left(\frac{n}{\log n}\right)^{3}}\right)$



P ✗

NP – decision version ✓

NP-hard ✗

RSA Encryption

KeyGen

p, q : random large primes

$e \in \{0, 1, \ldots, n-1\}$ s.t. $\gcd(e, \underbrace{(p-1)(q-1)}_{\text{euler's fn } \phi(n)}) = 1$

d s.t. $e \cdot d$ has remainder 1 mod $(p-1)(q-1)$

$\Rightarrow$ d is the multiplicative inverse of e

Bob keeps d private

publishes $N = p \cdot q$, e

Enc(N, e, m):

return $\boxed{m^e \bmod N}$ ← c

Dec(N, d, c):

return $c^d \bmod N$